2MA100 - Devoir Maison 2

Les exercices de ce devoir maison noté sur 20 points doivent être rendus au choix au format **Notebook** (`.ipynb`) ou **Script** (`.py`) en un ou plusieurs fichiers au plus tard le **7 mai 2021** à 20h00 sur moodle.

Attention Plagiat.

Les devoirs maisons doivent être codés et rendus de manière individuelle. A titre d'information, voici une définition du plagiat adaptée du Memento de l'Université de Genève :

Le plagiat consiste à insérer, dans un travail académique, des formulations, des phrases, des passages, des morceaux de code, des images, de même que des idées ou analyses repris de travaux d'autres auteurs, en les faisant passer pour siens.

En particulier, le copier-coller à partir de sources trouvées sur Internet ou sur des travaux d'autres étudiant·es sans citer les sources est considéré comme du plagiat et implique une note zéro. Le plagiat constitue également une tentative de tricherie sanctionnée par le règlement de l'université. La solution est d'indiquer dans vos devoirs tout de ce qui ne vient pas de vous en mentionnant les sources (page Internet, livres, autre étudiant·e,...). Tous les fichiers rendus seront analysés automatiquement avec un logiciel de détection des similarités (entre étudiant·es et depuis Internet).

Les modules Numpy et Matplotlib peuvent être utiles dans certains exercices :

```
import numpy as np
import matplotlib.pyplot as plt
```

Exercice 1 : Conjecture due à Euler

Euler a conjecturé en 1769 qu'au moins k puissances k-ième d'entiers strictement positifs sont nécessaires pour que la somme soit elle-même une puissance k-ième. En d'autres termes si $n \geq 2$, $k \geq 1$, $a_1, a_2, \ldots, a_n \geq 1$, et $b \geq 1$ sont des entiers tels que

$$\sum_{i=1}^{n} (a_i)^k = b^k \,.$$

alors nécessairement $n \geq k$. Cette conjecture a été refutée en 1966 par Lander & Parkin (doi:10.1090/S0002-9904-1966-11654-3) dans ce qui semble être le plus court article mathématique jamais écrit avec un contre-exemple pour $k = 5$:

$$27^5 + 84^5 + 110^5 + 133^5 = 144^5 \,.$$

Le but est de montrer que ce contre-exemple est le plus simple possible, dans le sens où il est l'unique contre-exemple avec $k \leq 5$ et $b \leq 144$.

À noter que la conjecture est trivialement vraie pour $k = 1$ et $k = 2$ vu que par hypothèse $n \geq 2$, donc nécessairement $n \geq k$.

a) Vérifier avec Python le contre-exemple ci-dessus.

b) Écrire une fonction `powers(bmax,k)` qui retourne l'ensemble de tous les entiers de de 1 à `bmax` élevés à la puissance `k`.

c) Vérifier qu'il n'existe pas de contre-exemple avec $k = 3$ et $b \leq 144$.

d) Écrire une fonction `combinaisons(liste,n)` qui pour une liste d'entiers `liste` et un entier `n` retourne la liste de toutes les combinaisons avec répétitions de `n` entiers de `liste` classés par ordre croissant. Par exemple `combinaisons([1,2,3,4],2)` doit retourner `[(1,1), (1,2), (1,3), (1,4), (2,2), (2,3), (2,4), (3,3), (3,4), (4,4)]`.

Indication : Utiliser une fonction récursive sur `n`.

e) Écrire une fonction `test(bmax,n,k)` qui pour trois entiers `bmax`, `n` et `k` donnés, itère sur toutes les combinaisons de `n` entiers retournés par `combinaisons` et vérifie si la somme de ces `n` entiers élevés à la puissance `k` est un entier présent dans la liste `powers(bmax,k)`. Utiliser cette fonction pour vérifier qu'il n'existe pas de contre-exemple à la conjecture d'Euler pour $k = 4$ et $b \leq 144$. Suivant la puissance de votre ordinateur, il est possible de choisir également $k = 5$ et donc de vérifier que le contre-exemple de l'introduction est bien le plus simple.

Pour $k = 5$, la méthode précédente consistant à itérer sur toutes les combinaisons est plutôt lente. Une méthode plus rapide consiste à observer que l'ensemble des sommes du type :

$$(a_1)^5 + (a_2)^5 + (a_3)^5 + (a_4)^5 \,,$$

peut s'écrire comme $S_1 + S_2$ où S_1 et S_2 sont des sommes de deux entiers à la puissance cinq.

f) Écrire une fonction `sum2(bmax,k)` qui retourne un dictionnaire ayant pour clefs les sommes $(a_1)^k + (a_2)^k$ avec la valeur associée (a_1, a_2) pour $0 \leq a_1 \leq a_2 \leq$ `bmax`. On prendra soin d'enlever l'élément trivial zéro du dictionnaire. Dans la construction du dictionnaire, s'assurer qu'il est défini de manière unique dans le sens où il n'existe pas une autre valeur possible pour une clef existante. Tester `sum2(300,5)` et `sum2(300,3)`.

g) Utiliser le dictionnaire construit précédemment pour déterminer l'ensemble des contre-exemples pour $k = 5$ et $b \leq 300$ en itérant sur tous les éléments de `power(bmax,5)` et de `sum2(bmax,5)`.

Indication : Une implémentation optimale prend au plus quelques secondes à s'exécuter.

Exercice 2 : Ensemble de Mandelbrot

L'ensemble de Mandelbrot est défini comme l'ensemble des points $c \in \mathbb{C}$ pour lesquels la suite de nombres complexes définis récursivement par $z_0 = 0$ et

$$z_{n+1} = z_n^2 + c \,,$$

est bornée. Il est possible de montrer que $c \in \mathbb{C}$ est dans l'ensemble de Mandelbort si et seulement si $|z_n| \leq 2$ pour tout entier n.

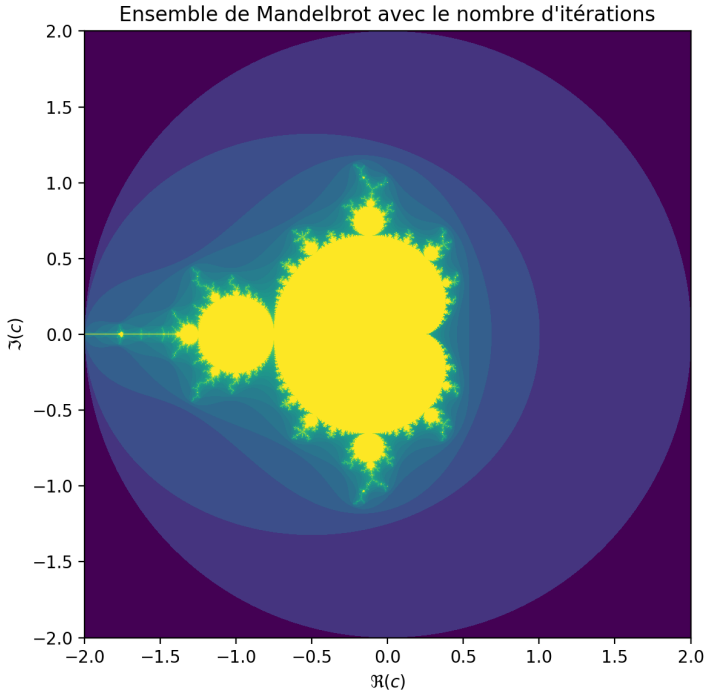
a) Écrire une fonction `mandelbrot(c)` qui vérifie si le point $c \in \mathbb{C}$ est dans l'ensemble de Mandelbrot de manière approximative en testant les cent premiers termes.

b) Tester la fonction précédente avec $c = 0$ et $c = 1 + i$. Qu'est-il attendu théoriquement ?

c) Écrire une fonction qui génère un tableau de taille $N \times N$ représentant l'ensemble $c \in \{x + iy : x \in [-2, 2] \text{ et } y \in [-2, 2]\}$ et qui retourne un tableau de booléens de taille $N \times N$ déterminant si le point associé est dans l'ensemble de Mandelbrot ou non.

d) En utilisant la fonction précédente avec $N = 101$, représenter graphiquement avec `imshow` une approximation de l'ensemble des points appartenant à l'ensemble de Mandelbort.

e) En adaptant les fonctions précédentes, représenter graphiquement le logarithme du nombre d'itérations nécessaires avant de ne plus satisfaire $|z_n| \leq 2$ au lieu d'un booléean.



f) [bonus] La méthode précédente a le désavantage de procéder séquentiellement au calcul pour chaque valeur de c, ce qui rend l'évaluation assez lente. Proposer une nouvelle implémentation permettant de calculer parallèlement toutes les valeurs en utilisant les indexages Numpy.

Exercice 3 : Théorème central limite

Le théorème central limite (aussi appelé théorème limite central, théorème de la limite centrale ou théorème de la limite centrée) établit la convergence en loi de la somme d'une suite de variables aléatoires vers la loi normale. Intuitivement, ce résultat affirme qu'une somme de variables aléatoires identiques et indépendantes tend (sous certaines conditions) vers une variable aléatoire gaussienne.

Voici une formulation de ce théorème :

Théorème.

Soit (X_n) une suite de variables aléatoires réelles indépendantes et de même loi admettant une espérance μ et un écart-type $\sigma \neq 0$. Soit $(\bar{X}_n)$ la suites définie par $\bar{X}_n = \dfrac{1}{n}\displaystyle\sum_{k=1}^{n} X_k$. Pour n assez grand, la loi de $\bar{X}_n$ peut être approché par la loi normale $\mathcal{N}(\mu, \frac{\sigma^2}{n})$.

L'objectif de cet exercice est de vérifier si ce théorème est valable pour différentes lois de probabilité :

- **Loi de Poisson** : distribution discrète sur $\mathbb{N}$, de paramètre λ, définie par :
$$\mathbb{P}(X = k) = \exp(-\lambda)\frac{\lambda^k}{k!}, \quad \forall k \in \mathbb{N}.$$
- **Loi normale** : distribution continue sur $\mathbb{R}$, de paramètres m et σ, définie par la densité :

$$\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-m)^2}{2\sigma^2}\right), \quad \forall x \in \mathbb{R}.$$

- **Loi de Cauchy** : distribution continue sur $\mathbb{R}$, de paramètres a et γ, définie par la densité :

$$\frac{1}{\pi\gamma\left(1+\left(\frac{x-a}{\gamma}\right)^2\right)}, \quad \forall x \in \mathbb{R}.$$

a) Définir une fonction `densite_loi_normale(x, mu var)` prenant comme un argument :

- `x` (tableau) : un tableau de nombres flottants
- `mu` (nombre flottant) : moyenne
- `var` (nombre flottant strictement positif) : variance σ^2

et qui renvoie la densité de la loi normale évaluée pour chaque nombre x dans `x` :

$$\mathcal{N}(\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right).$$

b) Regarder la documentation de la fonction `numpy.random.poisson` et générer 10 valeurs aléatoires selon une loi de Poisson de paramètre $\lambda = 2$.

c) Écrire une fonction `generer_echantillons_poisson(lam, N, M)` qui prend en arguments :

- `lam` (réel strictement positif) : paramètre λ pour la loi de Poisson
- `N` (entier strictement positif) : nombre d'expériences
- `M` (entier strictement positif) : nombre de variables aléatoires générées pour chaque expérience

et qui génère `N` expériences avec `M` variables aléatoires générées par expérience, et qui renvoie :

- (nombre flottant) : la valeur moyenne sur les `N * M` variables aléatoires générées,
- (nombre flottant) : l'écart-type sur les `N * M` variables aléatoires générées,
- (tableau) : vecteur numpy (*i.e.* un tableau de taille `(N,)`) où chaque élément est la moyenne des variables aléatoires d'une expérience.

Pour les valeurs `lam=2` et `N=10000`, exécuter `generer_echantillons_poisson(lam, N, M)` pour $M \in \{10, 100, 1000\}$ et sauvegarder les résultats dans des variables.

d) Pour chaque valeur de `M`, afficher la distribution du vecteur numpy contenant les moyennes de chaque expérience, ainsi que la distribution de la loi normale attendue si la théorème central limite est vérifié. On pourra utiliser la fonction `matplotlib.pyplot.hist` pour afficher l'histogramme d'un tableau, avec le paramètre `bins=50` pour fixer le nombre colonnes et `density=True` pour afficher une distribution de probabilité. On utilisera les moyennes et les écart-types empiriques renvoyés par la fonction `generer_echantillons_poisson` pour les paramètres de la loi normale. Choisir des valeurs pertinentes pour les valeurs limites de l'axe des abscisses. Faire une hypothèse sur la validité du théorème central limite pour la loi de Poisson.

e) Reprendre les questions **c)** et **d)** pour la loi normale. Pour cela utiliser la fonction `numpy.random.normal`, générer des moyennes pour `loc=2`, `scale=1`, `N=10000` et $M \in \{10, 100, 1000\}$. Tracer les histogrammes et faire une hypothèse sur la validité du théorème central limite pour la loi normale.

f) Reprendre les questions **c)** et **d)** pour la loi de Cauchy avec $a = 0$ et $\gamma = 1$. Pour cela utiliser la fonction `numpy.random.standard_cauchy`. Utiliser `bins=np.arange(-10, 10.1, 0.1)` et faire une hypothèse sur la validité du théorème central limite pour la loi de Cauchy.